॥वसुधैव कुटुम्बकम्॥

SYMBIOSIS INSTITUTE OF TECHNOLOGY , PUNE

Constituent of Symbiosis International (Deemed University),Pune

CASE STUDY REPORT ON

Case Study Centred on the 2019 WhatsApp–NSO Group/Pegasus incident

Subject: Cyber Security

Topic selected: End-to-End Encryption: Security Mechanism, Authentication and Non-Repudiation in Modern Communication(Unit 4)

Group No.: 15

By Group Members:

Prafful Kulkarni – 23070123099

Saanvi Mohapatra – 23070123110

Sai Sankar Jena – 23070123112

Subham - 23070123132

Faculty: Dr Mangal Singh

Department of Electronics & Telecommunication Engineering

B.Tech Batch 2023–27, Semester VII



Abstract

The most secure way of communicating over public cellular and IP networks is one of end-to-end encryption [cite: 3, 21]. Data confidentiality is achieved by using cryptographic transformations on the data content only at the nodes at the opposite ends of the communication over the network, to prevent an attack on the communications infrastructure, relays, carriers, and/or eavesdroppers [cite: 3, 12, 19]. With over two billion registered users around the world, WhatsApp's end to end encryption relies on Signal's Extended Triple Diffie-Hellman (X3DH) protocol for keys exchange and Double Ratchet protocol for message confidentiality and forward secrecy [cite: 3, 19, 20]. But they have been shown as lacking in lower layers of the network stack with respect to protection mechanisms in-use when it is used to encrypt. In this report, we report in detail on the end-to-end encryption of WhatsApp and focus mainly on the memory-based attacks that can potentially break the confidentiality of plaintext messages protected by Signal. We show how the implementation of WhatsApp can be used to mount a remote code execution attack on the targeted device, by causing a heap buffer overflow in libwhatsapp.so that occurs when it processes specially crafted Secure Real-time Transport Control Protocol (SRTCP) packets [cite: 23, 25, 30, 31]. With this flaw, an attacker would be able to steal the contents of the memory of the targeted WhatsApp running on iOS or Android, thus being able to read plaintext messages in the target's RAM that are not otherwise accessible. There is also a discussion on entity authentication, AES-GCM message integrity, deniability of content, and legal non-repudiation; a look at how these are handled in WhatsApp's defense-in-depth design principles, helps mitigate potential attacks against the encryption infrastructure. These range from memory safe Rust programs to process isolation within the operating system, fuzzing programs, and hardware isolation of cryptographic keys [cite: 7, 35, 37, 47].

Keywords:

WhatApp End to End Encryption, Signal Protocol, X3DH Key Exchange, Double Ratchet, CVE-2019-3568, Pegasus Exploit, VoIP Stack, SRTCP Heap Overflow, Deniability, Legal Non-repudiation, Memory Safety, Defense in Depth, Rust Programming [cite: 3, 19, 23, 26, 30, 37, 47].



1. INTRODUCTION

1.1 Evolution of Messaging Security and WhatsApp E2EE Adoption

The world of communication has evolved through a period of transformation characterized by the transition from using all unencrypted communication channels (such as SMS, early HTTP-based chat applications, and others) to the use of end-to-end-encrypted communication communities worldwide [3, 21]. In the early days of mobile communication, messages were transmitted through carrier networks in clear text and only protected on their way between the user's device and the carrier's main server [8, 17].

The standard TLS transit protects data from local eavesdropping on the wireless link, but it exposes messages' plaintext, multimedia content, and database content on central servers to operational inspection [8, 17]. Centralized architecture is a critical vulnerability as it creates risks of large-scale data leakage, insider threats, and government surveillance of messaging platforms [12, 26].

WhatsApp began collaborating with the Open Whisper Systems (Signal Technology Foundation) in 2014 to adopt the Signal Protocol for use in its multi-platform clients (Android, iOS, Windows Phones, desktop versions) [3, 21]. In April 2016, WhatsApp announced achieving End-to-End Encryption (E2EE) on a global scale and securing the one-to-one text messaging and group messaging, voice and video calls, media exchange, file transfer, and status updates features for more than 2 billion users [3]. WhatsApp's E2EE architecture allows devices to exchange only encrypted data with other devices and only decrypted data from trusted devices [3, 19].

The encryption and decryption happen exclusively inside the user's device, while WhatsApp's infrastructure is a ciphertext-only proxy. The system's security is founded on the secrecy of the session keys, which are exchanged only between the communicating devices, thus allowing only the participating devices to read the messages [3, 12].

.

1.2 Problem Statement: The Protocol-Endpoint Execution Dichotomy

Modern application-layer cryptographic protocols have formal cryptographic proofs, but there is a fundamental security dichotomy between the security guarantees of cryptographic protocols and the safety of the software implementation on the client hardware [12, 23]. Tools for cryptographic analysis (such as automatic symbolic verifiers ProVerif and CryptoVerif) reason about protocol's resistance to the classical adversary model of Dolev-Yao [12, 13]. The Dolev-Yao adversary model assumes that the network channel is insecure controlled by the



adversary, but the endpoints of communication are secure computing platforms not subject to attacks [12, 13].

The software stack of the client messaging application is significantly complex and large – tens of millions of lines of code written in low-level and unsafe programming languages such as C and C++ [cite: 23, 30, 37]. Incoming packets should pass through the network socket listeners and transport framing modules, media decoding and signaling parsing subsystems before reaching the cryptographic layer for processing and decrypting [cite: 23, 25, 30].

An arbitrary parsing subroutine in transport framing or media decoding module can have an unchecked overflow vulnerability in the memory buffer which can be exploited by an attacker. The adversary can craft malicious network packets which would be parsed in the client application in such a way that would violate the memory safety and arbitrary code execution (RCE) would be possible before any cryptographic processing of the packet [cite: 23, 30, 31].

After that, an attacker can perform arbitrary operations inside the memory space of the client application. In particular, with RCE, the adversary can read the memory of the client application and leak the plaintext of messages protected by End-to-End Encryption (E2EE), bypassing cryptography primitives [cite: 3, 12, 23, 26]. If the transport-layer E2EE is utilized in the application, the attacking code executed inside the client process can read the memory of the application process and exfiltrate the short-lived encryption keys and the content of the local sqlite databases directly from the memory [cite: 3, 12, 23, 26].

1.3 Scope, Methodological Boundaries & Evidence Sources

This is the case study of multi-disciplinary security analysis of WhatsApp's E2EE implementation, with the endpoint vulnerability vectors [cite: 3, 23, 28], and a detailed mathematical walk-through of the Diffie-Hellman key agreement and Double Ratchet algorithms, analysis of WhatsApp's 60-digit safety number fingerprint scheme, examination of post-quantum (PQXDH/ML-KEM-768) extensions, low-level C/C++ deconstruction of the recently-disclosed CVE-2019-3568 VoIP heap buffer overflow, and an assessment of the legal precedent set by WhatsApp LLC v. NSO Group Technologies Ltd. [cite: 3, 9, 10, 19, 20, 23, 28].

The report is methodologically bounded by the essay of qualitative evidence synthesis (QES) [cite: 23, 26, 28]. This is a secondary source engineering case study that is based on the official vulnerability information contained within the National Vulnerability Database (NVD) [cite: 3]



as well as cryptographic specification reviews conducted by Citizen Lab [cite: 25, 26, 28] and Check Point Research [cite: 23] and the sworn federal court filings of the U.S. District Court for the Northern District of California [cite: 26, 28]. No live zero-day exploits binaries or decompiling of commercial spyware samples [cite: 23, 28] were created. If there is no server telemetry information available for the proprietary server or the server telemetry information is confidential, the analysis will make a note of this [cite: 23, 28].

1.4 Distinction Between Cryptographic Deniability & Legal Non-Repudiation

An important difference between the notions of cryptographic deniability and legal non-repudiation is that cryptographic deniability is a technical distinction [cite:7, 13, 21]. Message Authentication Codes (MACs) are computed with shared symmetric keys and are not using asymmetric digital signatures such as RSA or ECDSA signatures as the modern secure messaging systems like WhatsApp and Signal do, and are used for per-message authentication [cite:3, 19, 20]. Both conversation partners share a symmetric key, which is to compute MAC tags, and either of them is mathematically able to generate a valid MAC tag for an arbitrary transcript of the conversation [cite:19, 20].

Bob can plausibly claim that Alice forged the transcript using her copy of the shared symmetric key [cite:19, 20]: it's called off-the-record cryptographic deniability [cite:19, 20].

Legal non-repudiation is a concept defined by law as having recorded information which an agent can not deny the authenticity of a particular signed statement/transaction [cite:6, 7, 44]. To achieve legal non-repudiation it needs asymmetric digital signatures bound to hardware security modules (HSMs), trusted public key infrastructure (PKI) identity certificates and timestamping authorities to trustworthy [cite:6, 7, 44]. Secure non-repudiation is one of the property rights of an electronic message that messaging systems which use privacy protection refuse to offer.Messaging systems that rely on privacy protection explicitly object to the legal non-repudiation property, to prevent a leak of the message transcript.

But with respect to law and court of law, who's the author can be decided by the use of contextual corroborating evidence such as when a physical device (phone) is seized and metadata, carrier cell tower logs, and forensic images of an acquired device are examined: cryptographic deniability did not shield one from legal liability when one's phone was physically seized [cite:26, 28].



2. WHATSAPP E2EE ARCHITECTURE & CRYPTOGRAPHIC MECHANICS

2.1 Untrusted Relay Server Model & Operational Data Flow

1. WhatsApp's worldwide messaging network is based on an untrusted relay server design [cite: 3, 21, 24]. The central 'server' part coordinates the user directories' registration, provisioning of public key bundles, phone number validation, off-line message queues, push notification 'relays' and routing of packets of transport [cite: 3, 21]. The relay server architecture uses cryptographic security to allow the complete separation of access to plaintext and derivation of session keys [cite: 3, 19].
2. The end-to-end data path for Alice to send an encrypted message to Bob through WhatsApp's encrypted messaging layer is then traversed in six main processing stages [cite: 3, 11, 19, 20]:
3. Prekey Retrieval: If Alice and Bob don't already have an active cryptographic session, then Alice asks Bob's WhatsApp client application to check the public prekey bundle stored in the WhatsApp server directory [cite: 3, 20].
4. Session Initialization (X3DH): Alice uses her client to check the signed prekey signature from Bob, creates an ephemeral key pair, and computes the X3DH key agreement computation to obtain a master shared secret (S) [cite: 3, 20].
5. Payload Encryption: Alice starts her Double Ratchet engine, generates a per-message ephemeral key (MKi), and encrypts the plaintext content of her message using AES-256-GCM, producing the content of each message in the message body along with a 128-bit authentication tag (19, 3, 4).
6. The application-layer encrypted payload envelope (which contains the ciphertext, authentication tag, initialization vector, and information for the "ratchet") is encapsulated into a Noise Protocol framework socket frame that is sent over TCP or UDP to WhatsApp's routing servers [cite: 3, 11].
7. Server Relay Processing: WhatApp's route servers read and interpret unencrypted headers informing them of the recommended account ID numbers for the recipient; If the recipient Bob is not online, the message envelope will be held in the queuing area until he is next connected, or they will immediately relay it to Bob's active push connection [cite: 3, 21?]. The server server does not support decryption of the message payload, or it is not configured to do so [cite: 3].



8. Client Decryption & Verification: Bob receives the binary message envelope, performs the ratchet header parameters to update the local DH/KDF ratcheting state, decrypts the tag of the AEAD and then displays the decrypted plaintext [cite: 3, 4, 19].

2.2 Extended Triple Diffie-Hellman (X3DH) Mathematical Key Agreement

In order to support asynchronous session establishment, supporting the sending of an encrypted message, even if Bob is offline at the moment of composition, WhatsApp implements the Extended Triple Diffie-Hellman (X3DH) protocol [cite: 3, 20]. For every client using WhatsApp there are four key pairs generated and stored which are computed from the Montgomery curve Curve25519 (a 128-bit secure curve) or Ed25519 [cite: 3, 14, 20]:

- Identity Key (IK): A key pair that is long-term and is specific to the user account that it was created for [cite: 3, 20].
- Signed Prekey (SPK): A medium term key pair that is replaced periodically, and signed by the user's Identity Key to prevent server side key forgery [cite:3,20].
- One-Time Prekeys (OPK): Prekeys that are different for each session, uploaded to the server and consumed as the session begins [cite: 3, 20].
- Ephemeral key (EK): One time key pair of type generated by the session initiator (Alice) [cite: 3, 20].

When Alice requests a cryptographic session with Bob she downloads from the server a prekey bundle (IKB,SPKB,SigB,OPKB) of Bob [citations 3, 20]. Alice checks identity key of Bob, IKB [cite: 20] (which is public) using his public identity key to verify SigB. Alice then generates her ephemeral key pair, EKA and then computes (up to) four Elliptic Curve Diffie-Hellman (ECDH) computations [cite: 3, 20]:

DH1 = ECDH(IK_A, SPK_B) [Mutual Identity Authentication]
DH2 = ECDH(EK_A, IK_B) [Forward Secrecy Alice's IK Leakage]
DH3 = ECDH(EK_A, SPK_B) [Forward Secrecy against Ephemeral Leakage]
DH4 = ECDH(EK_A, OPK_B) [One-Time Prekey Forward Secrecy] [cite: 20]

These ECDH calculations then generate a set of bytes, which are concatenated in a fixed order and fed into HKDF-SHA256 to derive the master shared secret (S) [cite: 15, 20]:

Master Shared Secret (S) = HKDF-SHA256(DH1 || DH2 || DH3 || DH4) [cite: 15, 20]



2.3 The Double Ratchet Algorithm: State Machine & Key Evolution Loops

The Double-Ratchet algorithm [cite: 3, 19] guarantees confidentiality of future messages, per-message forward security and break-in recovery after the initial keys have been established via X3DH. This algorithm has two intertwined cycles of key derivation that are executed at the same time [cite: 19]:

Symmetric-Key KDF: Ratchet: Every time any message is sent, the loop is executed repeatedly for each message [cite: 19]. There is a KDF Chain Key (CK) for each sending and receiving directions [cite: 19]. Whenever a specific message is to be sent, CKi is passed to HKDF-SHA256 in order to derive two distinct outputs: one-time Ephemeral Message Key (MKi) and the Next Chain Key (CKi+1) [cite: 15, 19]. This is to ensure envelope encryption with MKi while zeroing it out after it has been used to encrypt one message to give per-message forward secrecy [cite: 19, 24].

Diffie-Hellman (DH) Ratchet: This loop is interactive, and is executed at every iteration of the reversion of the conversation to the respective party [cite: 19]. The ratchet public key (Ratchet_DH) of the sender at that time is sent in the message headers [cite: 19]. Whenever Bob receives a message from Alice with a new ratchet public key, he does a DH ratchet step, computes the new top-level Root Key (RK), derives new keys for sending and receiving chains and generates a new local ratchet key pair (Ratchet_DHB) [cite: 19].

If an adversary is able to extract one of the two endpoints' chain keys from memory, then the two loops will guarantee that future communications between the two endpoints will still be safe, since the compromised endpoint sends or receives any message with a new DH ratchet step (Post-Compromise Security / Break-in Recovery) [cite: 19, 21].

॥वसुधैव कुटुम्बकम्॥

Table 1: CVE-2019-3568 Incident Timeline

Date / Period	Event	Technical Significance	Response / Outcome	Source
2016–2018	Citizen Lab investigations on NSO Group Pegasus spyware	Identified commercial spyware targeting dissidents, journalists across 45 countries	Initiated global forensic monitoring of NSO C2 infrastructure	Citizen Lab [cite: 26, 27]
May 2019	Active exploitation of CVE-2019-3568 in WhatsApp VoIP stack	Unauthenticated zero-click SRTCP packet injection causing heap buffer overflow in libwhatsapp.so	WhatsApp patched vulnerability, updated server infrastructure, and deployed client updates	NVD / WhatsApp [cite: 23, 24, 25]
May 13, 2019	WhatsApp public security advisory and user notification	Formal disclosure of CVE-2019-3568 heap overflow; targeted notifications sent to ~1,400 affected users	Prompted global app updates; forced attacker infrastructure to decommission active exploit vector	WhatsApp Security [cite: 24, 28]
October 2019	WhatsApp LLC & Meta file lawsuit against NSO Group	Filed civil complaint in U.S. District Court (N.D. Cal.) alleging CFAA (§1030) and CDAFA (§502) violations	Established precedent of major platform suing commercial surveillance vendor on behalf of users	Federal Court Filings [cite: 28]
November 2021	Ninth Circuit Court of Appeals ruling on foreign sovereign immunity	Court rejected NSO's claim of derivative foreign sovereign immunity under FSIA	Permitted WhatsApp's CFAA civil lawsuit to proceed toward discovery in U.S. federal court	9th Circuit Ruling [cite: 29]
2023–2025	U.S. Supreme Court cert denial; District Court sanctions & liability rulings	Supreme Court denied cert (Jan 2023); District Court issued evidentiary sanctions & granted summary judgment	Held NSO Group liable for CFAA, CDAFA, and breach of contract violations	District Court Orders [cite: 28, 29]

2.4 Consolidated Cryptographic Suite & Security Objectives Matrix

WhatsApp uses standard cryptographic primitives in its E2EE protocol pipeline [cite: 2, 3, 4, 14, 15, 16]. These primitives and their security goals and boundaries are summarized in Table 2:



Table 2: Comprehensive Cryptographic Primitives & Core Security Objectives in WhatsApp E2EE

Cryptographic Primitive	Standard / Specification	Core Operational Function in WhatsApp	Security Objective & Operational Boundary
Curve25519	RFC 7748 / Bernstein [cite: 14]	ECDH key agreement across X3DH handshakes and interactive DH ratchets [cite: 3, 14, 20].	128-bit security level; constant-time implementation immune to timing side-channels [cite: 14]. Vulnerable to quantum computers [cite: 8].
Ed25519	RFC 8032 / FIPS 186-5 [cite: 6, 14]	Digital signatures for identity key authentication and prekey binding [cite: 3, 20].	Provides entity authentication; prevents server-side prekey substitution attacks [cite: 3, 13, 20].
AES-256-GCM	FIPS 197 / NIST SP 800-38D [cite: 2, 4]	Symmetric message payload encryption and AEAD authentication tag calculation [cite: 3, 4].	Guarantees payload confidentiality and ciphertext tampering detection [cite: 4, 42]. Bypassed if endpoint RAM is read [cite: 23].

HKDF-SHA256	RFC 5869 / FIPS 180-4 [cite: 5, 15]	Extract-and-Expand key derivation across master secrets, root keys, and chain keys [cite: 15, 19].	Provably secure pseudorandom key expansion; separates key contexts cleanly [cite: 15].
HMAC-SHA256	FIPS 198-1 / RFC 2104 [cite: 5]	Per-message MAC tag generation and symmetric KDF chain ratcheting [cite: 3, 19].	Provides symmetric message authentication and off-the-record cryptographic deniability [cite: 19, 20].
ML-KEM-768 (PQXDH)	NIST FIPS 203 / Kyber-768 [cite: 9, 10]	Post-quantum lattice key encapsulation combined with Curve25519 ECDH [cite: 9, 22].	Protects traffic against future retrospective decryption by quantum computers [cite: 8, 9, 22].

2.5 Security Fingerprints, 60-Digit Safety Numbers, and Out-of-Band Key Verification

To protect the users from the Man-in-the-Middle (MitM) key substitution during initial X3DH setup by a compromised relay server, WhatsApp provides an out-of-band identity verification mechanism called Security Code / Safety Numbers [cite: 3, 12, 21]. The 60-digit numeric fingerprint is generated by WhatsApp from the lexicographically sorted public Identity Keys (IKA and IKB) of both the two parties of the chat [cite: 3].

The 60 digit fingerprint is computed from the concatenation of the identity keys, hashing it with SHA-512 followed by parsing the resulting 512 bit-long hash into 5 12-digit blocks of numbers



with a protocol version prefix [cite: 3]. Key binding is done by face-to-face scanning of a 2D QR code, or checking the 60 digit numeric string via an alternative channel which is already established to be reliable, like PGP signed email or in person [cite: 3, 21]. If WhatsApp server tries to do an active key substitution attack, the strings generated, would not match with the strings generated by the interceptor [cite: 3, 12]. But empirical usability studies show that more than 90% of non-technical users don't pay attention to the automatic key-change notification warnings, thus introducing a significant human factor security risk [cite: 3, 12].

2.6 Post-Quantum Extended Key Agreement (PQXDH & ML-KEM-768 Integration)

The future quantum resilient algorithm for the encryption of the WhatsApp traffic is possible as it is now possible to collect and store this traffic for subsequent decryption by future quantum computers [cite: 8, 9, 22].

The current Signal Protocol specifications utilize the Post-Quantum Extended Diffie-Hellman (PQXDH) for the resistance to the quantum interception purposes [cite: 9, 22]. PQXDH specification includes the hybrid key exchange algorithm of the classic Curve25519 ECDH and lattice Key Encapsulation Mechanism such as ML-KEM-768 (NIST FIPS 203 standard, Kyber-768 based) [cite: 9, 10, 22]. The PQXDH session is established by making a classical ECDH operation and a ML-KEM encapsulation call [cite: 9, 22]. The hybrid master shared secret is computed as:
Hybrid Shared Secret = HKDF-SHA256(DH1 || DH2 || DH3 || DH4 || ML-KEM_SS) [cite: 9, 15, 22]

3. METHODOLOGY & EVIDENCE TRIANGULATION

3.1 Secondary-Source Qualitative Case Study Design

This report employs a qualitative, evidence-based cybersecurity case-study method specifically adjusted for the needs of analyzing zero-day exploitation and commercial surveillance practices [cite: 23, 26, 28]. The technical, operational, and legal limitations to direct dynamic analysis of the offensive cyberespionage tools (such as Pegasus) by NSO Group and other private security firms are particularly challenging: the software is restricted by the licensing policy, obfuscated and self-removing upon detection, and protected by the state-level defense secrecy laws [cite: 26, 27, 28].



3.2 Four-Tier Evidence Triangulation Framework

1. In order to ensure that the technical arguments presented are thoroughly supported, and in order to avoid confirmation bias, in this report we cross-reference all technical claims with the sources from all four levels as described in the Four-Tier Evidence Triangulation below [cite: 3, 19, 23, 25, 26, 28]:
2. For the vulnerabilities themselves, we rely on official repositories and advisories such as the NVD CVE database listings (CVE-2019-3568), WhatsApp Security Advisories and the MITRE Common Weakness Enumeration (CWE-120, CWE-787) [cite: 23, 24, 30, 31].
3. For the requirements of the security protocols used by WhatsApp and Signal, we rely on the public specifications literature from the companies which develop and maintain the protocols, notably the NIST FIPS 203,204 standard [cite: 3, 9, 10, 12, 19, 20]
4. Finally, for the technical analysis of the attack chains themselves, we rely on technical deep-dive analyses by Check Point Research and by Citizen Lab in their independent audits of the Pegasus spyware [cite: 25, 26, 27]
5. The technical capabilities and attack surface of Pegasus are then discussed in detail in the court documents of WhatsApp LLC v. NSO Group Technologies Ltd., currently under jurisdiction of the Ninth Circuit Court of Appeals, Northern District of California [cite: 28, 29].

4. CASE STUDY: THE 2019 WHATSAPP ZERO-CLICK EXPLOIT (CVE-2019-3568)

4.1 Incident Context, Discovery Timeline & Global Impact

In May of 2019, WhatsApp released a security advisory announcing the CVE-2019-3568, an unauthenticated remote code execution heap overflow bug in WhatsApp for android, WhatsApp for iOS, WhatsApp Business, and WhatsApp Enterprise clients [cite: 23, 24]. Over 1,400 targeted phones were used worldwide by a commercial exploit for sale by NSO group to install Pegasus spyware, including human rights lawyers, investigative journalists, dissidents and government officials [cite: 26, 27,28].



4.2 WhatsApp VoIP Stack Architecture & Real-time Media Signaling

The protocols used via WhatsApp's VoIP protocol are two different standard streaming protocols. The voice channel is streamed in packets using Real-Time Transport Protocol (RTP) and a Secure Real-Time Transport Control Protocol (SRTCP) is employed to control sessions and estimate jitter and for signaling [cite: 23, 25]. Real time media stacks have extremely low latency requirements. They must receive incoming streams of UDP packets, start up jitter buffers and complete signaling handshake prior to accepting or rejecting an incoming call [cite: 23, 25].

4.3 Low-Level Binary Analysis: Heap Buffer Overflow in `libwhatsapp.so`

CVE-2019-3568 falls under the Common Weakness Enumeration (CWE) of Buffer Copy Without Checking Size of Input (CWE-120) and Out-of-bounds Write (CWE-787) [cite: 23, 30, 31]. The problem was in the native C/C++ compiled shared object library “libwhatsapp.so” which parses and understands SRTCP signaling frames [cite: 23, 25].

If the WhatsApp client receives an incoming VoIP call, an SRTCP signaling stream will be sent directly to client socket listener, on the same UDP socket [cite: 23, 25]. The vulnerable parsing subroutine read the 8-bit length header field from the received packet, and passed the length field straight into the length argument of a memory copy operation (like memcpy) to a fixed-size buffer that had been allocated on the process heap [cite: 23, 25, 30]:

```
// Vulnerable Parsing Subroutine in libwhatsapp.so
void parse_srtcp_control_frame(uint8_t* socket_buffer, size_t recv_bytes) {
char heap_dest_buffer[256]; // Fixed 256-byte heap allocation
uint8_t srtcp_length = socket_buffer[2]; // Unsanitized 8-bit length header
// MISSING BOUNDS CHECK: if (srtcp_length > 256) return;
memcpy(heap_dest_buffer, socket_buffer + 4, srtcp_length); // HEAP OVERFLOW}
```

The attacker sent a malformed SRTCP packet that contains an over-sized length header field to cause memcpy to copy binary data past the 256-byte heap allocation boundary [cite: 23-25, 30]. Corrupts adjacent chunks of heap memory that are out-of-bounds, overwriting object metadata, vtable pointers and function pointers [cite: 23, 25, 31].

The vulnerables SRTCP parsing routine was automatically run when receiving a socket, so the exploit was able to gain "zero-click" Remote Code Execution (RCE) [cite: 23,25,26]. The target did not have to respond to the call, click any link or open any file [cite: 23, 26]. The attack



shellcode used to block WhatsApp user interface call ringers, delete call logs and hide UI notification alerts [cite: 25, 26] to conceal the attack via WhatsApp's user interface threads.

4.4 Pegasus Delivery, Privilege Escalation & Volatile RAM Exfiltration

This research paper focuses on WhatsApp's vulnerability for surveillance and data exfiltration on mobile devices by the help of Pegasus.

Once RCE is obtained on the WhatsApp app, the first-stage shellcode served as a stager to communicate with C&C servers hosting second-stage modules of spyware (see references 25, 26 and 27). Pegasus was killed on C2 servers located at remote computers, using local OS kernel privilege escalation (via unpatched Linux kernel or iOS WebKit/kernel vulnerabilities), to escape the sandbox (see references 26,27,35,36).

With kernel or root level privileges, Pegasus was able to create a long term footprint on the victim device (see references 26,27). The malware was able to intercept WhatsApp messages which are encrypted, as it was able to extract them directly from the process RAM, before they were sent, received or decrypted, by inserting hook subroutines (injecting DLLs) into system libraries (see references 23,26,27). Pegasus was also able to exfiltrate contact lists, SMS messages, location data, photos and real-time microphone and camera data (see references 26,27).

4.5 Boundary Invalidation Theorem: Why Protocol Security Fails on Broken Endpoints

The CVE-2019-3568 incident proves that the Boundary Invalidation Theorem of secure communications exists: Math channel security becomes completely null in case of client binary execution protection is broken [cite: 3, 12, 23]. During the implementation of the Signal Protocol, it has a mathematical manner that any eavesdropper on the middle network will not be able to see anything other but ciphertext, encrypted on a secure level [cite: 3, 12, 23]. However, the attacker managed to hijack the process at the point of presentation, achieving direct access to plaintext information in the display layer memory in RAM without having to traverse the cryptographic abstractions of the protocol first [cite: 12, 23, 26].



5. PREVENTIVE MEASURES & DEFENSIVE ENGINEERING HARDENING

5.1 Memory-Safe Systems Programming (Rust Migration)

A technical root cause of CVE-2019-3568 was using legacy C/C++ parsing libraries that are not memory-safe to process network data without validation. Citing the use of legacy, memory-unsafe C/C++ parsing libraries to process network data without validation, was the technical root cause of CVE-2019-3568. The best defense offered by secure messaging platforms is to use a memory-safe language for network parsers, media decoders and protocol engines such as Rust [cite: 37]. Rust gives compile-time memory safety and ownership, and borrowing guarantees, eliminating spatial memory corruption (buffer overflows, out-of-bounds writes) and temporal memory corruption (use-after-free, double free), without relying on garbage collection [cite: 30, 31, 37].

5.2 Input Validation, Bounds Verification & Coverage-Guided Fuzzing

In the places where legacy native C/C++ code needs to stay, it is imperative that input sanitization pipelines are enforced [cite: 30, 33]. All fields of the length, offset and count of the incoming packet header must be checked against tight limits before entering memory allocation and/or copy routines: if (srtcp_len > MAX_ALLOWED) return PARSE_ERR; [cite: 30, 33]. In addition, media parsers need to be continuously fuzzed to find parsing bugs in edge cases, before release, with the help of tools such as libFuzzer or AFL++. [cite: 33, 47]

5.3 OS Sandboxing, Least Privilege & System Call Filtering

OS-level process isolation [cite: 35, 36] is a way to reduce the blast radius of a process compromise for messaging platforms. High-risk processing networks should make use of network parsers running inside isolated sandbox processes that are limited through the use of OS system call filters (e.g., Linux seccomp-bpf filters, iOS App Sandboxing) [cite: 35, 36]. Parsed threads should not have access to the file system, microphone/camera devices and database sockets [cite: 35, 36]. Also, you should make use of mlock to store persistent identity private keys in hardware security enclaves (Android StrongBox or Apple Secure Enclave) which does not allow volatile key paging to disk [cite: 7, 35, 36].



5.4 Defensive Engineering Control Matrix

Table 3 shows the defensive engineering controls needed to make a messaging system invulnerable:

Table 3: Defensive Engineering Control Matrix for Secure Messaging Platforms

System Risk Vector	Control Priority	Recommended Defensive Mechanism	Verification & Testing Framework
Unauthenticated Packet Parsing Zero-Click RCE	Critical / Highest	Migrate network parsing engines to Rust/Go; enforce explicit bounds checking [cite: 30, 33, 37].	Continuous coverage-guided fuzzing (libFuzzer/AFL++) & AddressSanitizer (ASan) [cite: 33, 47].
Process Memory Escalation & Sandbox Escape	Very High	Restrict media parsers inside isolated sandboxes using seccomp-bpf filters [cite: 35, 36].	Sandbox boundary penetration testing & privilege reviews [cite: 34, 35].
Volatile Key Scraping from RAM	High	Lock session keys via mlock; zeroize key memory immediately post-use [cite: 7, 19].	Automated volatile RAM heap inspection post-decryption [cite: 7, 34].

Post-Quantum Traffic Harvesting	Medium-High	Deploy PQXDH hybrid handshakes combining Curve25519 with ML-KEM-768 [cite: 9, 10, 22].	Automated symbolic protocol verification in ProVerif [cite: 12, 22].

6. RESULTS, DISCUSSION & UNCERTAINTY ASSESSMENT

6.1 Critical Analytical Findings

1. When one looks at protocol specifications, vulnerability disclosures and forensic reports cross-wise one can draw three major conclusions [cite: 3, 12, 23, 26, 28]; (1): Cross-analysis of protocol specifications, vulnerability disclosures and forensic reports leads to three major conclusions (2): (3):
2. E2EE Transport Integrity Remained Intact: WhatsApp's usage of the Signal Protocol during this attack didn't get compromised [cite: 3, 23]. Payloads in motion were protected from being stolen by relay eavesdropping with success using Double Ratchet and X3DH algorithms [cite: 3, 19, 20].
3. Successfully exploited native C/C++ libraries that are used for parsing SRTCP without verification to subvert protocol defenses before protocol is verified (see [cite: 23, 25, 30]). Application execution layer [cite: 12, 23] was corrupted due to memory corruption, which invalidated confidentiality.
4. Encryption Does Not Equal Non-Repudiation: Encryption channels provide entity authentication, cryptographic non-repudiation, but do not provide legal non-repudiation [cite: 7, 13, 21]. Even if the assumption is made that the messages are humans, then endpoint compromise is a problem [cite: 23, 28].



6.2 Telemetry Gaps, Evidence Quality & Exploitation Uncertainties

It is essential to recognize the difference between what has been established and the limitations due to the lack of proper telemetry when analysing zero day incidents [cite: 23, 26, 28]. While court documents prove WhatsApp sent patch messages to some 1,400 user accounts, there are no public records stating the percentage of successful infections [cite: 26, 28]. It is worth mentioning that the attack chain consists of several different stages such as attack targeting, first r.c.e., the escape from the sandbox, the persistent installation of the espionageware [cite: 26, 35]. For instance, the targeted devices were attacked with initial exploits that caused ASLR randomization on a heap buffer overflow, but which did not drop the Pegasus malware [cite: 25, 26, 35]. Lastly, public forensic reports only include the devices under consideration which leads to selection bias [cite: 26, 28].

7. LEGAL, ETHICAL, AND ACCOUNTABILITY CONTEXT

7.1 WhatsApp LLC v. NSO Group Litigation & CFAA Rulings

In the fall of 2019, according to WhatsApp LLC's and its parent company, Meta Platforms Inc.'s complaint, WhatsApp LLC and the parent company filed a lawsuit against NSO Group Technologies Ltd. in the United States Northern District Court (WhatsApp LLC v. NSO Group Technologies Ltd., No. 4:19-cv-07123) [cite: 28] alleging that the company's agents violated Computer Fraud and Abuse Act (18 U.S.C. § 1030) and California Comprehensive Computer Data Access and Fraud Act (Cal. Using fake SRTCP packets and sending them down the whatsapp servers (Penal Code § 502), it was possible to run arbitrary code on the target phones [cite: 28].

The defendant moved to dismiss the case on the ground of foreign sovereign immunity. Since NSO Group's services were used by governmental entities, it was claimed that WhatsApp's lawsuit was not subject to the Court's jurisdiction [cite: 28, 29]. The Ninth Circuit has denied the motion in November 2021, on the basis of a precedent that private parties who sell surveillance devices to sovereign states do not have universal immunity [29]. In January 2023, the US Supreme Court declined to consider the case, and that could mean that WhatsApp's CFAA claims may be allowed to go to trial.



7.2 Coordinated Vulnerability Disclosure & Global Policy Impact

The WhatsApp zero-day event had a significant impact in the policies of the vulnerability disclosure as well as commercial surveillance regulation around the world [cite: 24, 38, 45]. This enabled the triggering of multiple regulatory responses including that of the US Department of Commerce adding NSO Group to the Entity List for malicious cyber activities [cite: 28, 38, 45]. The incident served to demonstrate that software vendors are required to have clearly defined Coordinated Vulnerability Disclosure (CVD) programs, provide detailed advisories, automatically apply their client software, updates, and actively notify their target users in the case of targeted exploitation [cite: 24, 38, 47].

8. CONCLUSION AND FUTURE SCOPE

8.1 Synthesis of Strategic Lessons

WhatsApp End-To-End Encryption and CVE-2019-3568 zero-click incident analysis is a reminder that the current secure messaging platforms is the product of holistic system engineering approach [cite: 3, 12, 23, 47], and the de-facto standard for digital privacy is end-to-end encryption to protects actual content of the payload from relay servers, ISPs and eavesdroppers between two parties [cite: 3, 12, 19], the security of the protocols is only to up to binary interface of the client application [cite: 12, 23], an untrusted network can have memory corruption defects in their parsers that makes state of the art cryptographic algorithms practically useless for out-of-band RAM exfiltration [cite: 23, 25, 30]..

8.2 Actionable Engineering Mandates for Secure Messaging Platforms

1. To defend against zero-click endpoint attacks to communication platforms, software engineering teams must be aware of 7 architectural mandates [cite: 7, 30, 33, 35, 37, 47]:
2. Include Memory Safe Languages: require untrusted network parsing logic, media decoders and VoIP subroutines to be re-written in a memory-safe language (Rust) to prevent memory corruption [cite: 30, 37].
3. Strict Input Sanitization and Length Check on all Network Length Headers: before calling any memory allocation call explicitly check all the length fields on the network headers [cite: 30, 33].

4. Coverage-Guided Fuzzing: Integrate fuzzing pipelines (libFuzzer/AFL++) in the CI/CD pipeline and detect parsing bugs in advance of deployment [cite: 33, 47].
5. Isolate High Risk Subroutines: Only allow sandbox processes to perform VoIP media parsing through system call filters (seccomp-bpf) [cite: 35, 36].
6. Implementing volatile memory-locking and immediate zeroization post use for memory keys while storing long term ones in hardware enclaves [cite: 7, 35].
7. Switch to PQXDH: Use ML-KEM-768 with Hybrid Post-Quantum Handshakes for future quantum safe traffic protection [cite: 9, 10, 22].
8. Continue Emergency Patch Deployment: automate and silently deploy client updates to minimize exposure to vulnerabilities following disclosures [cite: 38, 47].

8.3 Future Research Scope

Further work is needed to validate out-of-band key verification techniques (e.g., CompCert and F*), identify memory safety hardware architectures (e.g., ARM Capability Hardware Enhanced RISC Instructions - CHERI) that can protect the client device from zero-day attacks [cite: 12, 22, 37, 47].

9. REFERENCES

[1] National Institute of Standards and Technology (NIST), "Recommendation for Key-Derivation Methods in Key-Establishment Schemes," *NIST Special Publication 800-56C Rev. 2*, 2020.

[2] NIST, "Advanced Encryption Standard (AES)," *FIPS PUB 197*, updated 2023.

[3] WhatsApp LLC, "WhatsApp End-to-End Encryption Technical Whitepaper," WhatsApp Security Documentation, 2021.

[4] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," *NIST Special Publication 800-38D*, 2007.

[5] NIST, "Secure Hash Standard (SHS)," *FIPS PUB 180-4*, 2015.

[6] NIST, "Digital Signature Standard (DSS)," *FIPS PUB 186-5*, 2023.

[7] NIST, "Security Requirements for Cryptographic Modules," *FIPS PUB 140-3*, 2019.

[8] NIST, "Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations," *NIST Special Publication 800-52 Rev. 2*, 2019.

[9] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," *FIPS Publication 203*, National Institute of Standards and Technology, Gaithersburg, MD, 2024.



[10] NIST, "Module-Lattice-Based Digital Signature Standard," *FIPS Publication 204*, National Institute of Standards and Technology, Gaithersburg, MD, 2024.

[11] T. Perrin, "The Noise Protocol Framework," *RFC 9420*, 2023.

[12] C. Cremers, M. Horvat, J. Hoyland, and S. Scott, "A Comprehensive Symbolic Analysis of the Signal Protocol," in *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, Paris, France, 2017.

[13] M. Bellare and P. Rogaway, "Entity Authentication and Key Distribution," in *Advances in Cryptology – CRYPTO '93*, Springer, 1993.

[14] D. J. Bernstein, "Curve25519: New Diffie-Hellman Speed Records," in *Public Key Cryptography – PKC 2006*, Lecture Notes in Computer Science, vol. 3958, Springer, Berlin, 2006.

[15] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," *RFC 5869*, Internet Engineering Task Force, 2010.

[16] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," *RFC 8439*, Internet Engineering Task Force, 2018.

[17] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," *RFC 8446*, Internet Engineering Task Force, 2018.

[18] NIST, "Guide to Computer Security Log Management," *NIST Special Publication 800-92*, 2006.

[19] M. Marlinspike and T. Perrin, "The Double Ratchet Algorithm," Signal Specification, Revision 1, 2016.

[20] M. Marlinspike and T. Perrin, "The X3DH Key Agreement Protocol," Signal Specification, Revision 1, 2016.

[21] Signal Technology Foundation, "Signal Protocol Specifications and Cryptographic Documentation," Signal.org, 2023.

[22] K. Bhargavan, C. Jacomme, F. Kiefer, and R. Schmidt, "Formal Verification of the PQXDH Post-Quantum Key Agreement for End-to-End Messaging," in *Proceedings of the USENIX Security Symposium*, 2024.

[23] National Vulnerability Database, "CVE-2019-3568 Detail," National Institute of Standards and Technology (NIST), May 2019.

[24] WhatsApp LLC, "Security Advisory: CVE-2019-3568 - Buffer Overflow in VoIP Stack," Facebook Security Bulletins, May 13, 2019.



[25] Check Point Research, "The NSO WhatsApp Vulnerability—This Is How It Happened," Technical Analysis Report, May 14, 2019.

[26] The Citizen Lab, "The Million Dollar Dissident: NSO Group's Pegasus Spyware," Munk School of Global Affairs and Public Policy, University of Toronto, 2016–2019.

[27] The Citizen Lab, "Hide and Seek: NSO Group's Pegasus Spyware Actively Operating in 45 Countries," Research Report No. 113, 2018.

[28] U.S. District Court for the Northern District of California, *WhatsApp LLC et al. v. NSO Group Technologies Ltd. et al.*, Case No. 4:19-cv-07123-PJH, Civil Filings and Orders, 2019–2024.

[29] U.S. Court of Appeals for the Ninth Circuit, *WhatsApp LLC v. NSO Group Technologies Ltd.*, 17 F.4th 930 (9th Cir. 2021), cert. denied, 143 S. Ct. 562 (2023).

[30] MITRE Corporation, "CWE-120: Buffer Copy without Checking Size of Input (Classic Buffer Overflow)," Common Weakness Enumeration, 2019.

[31] MITRE Corporation, "CWE-787: Out-of-bounds Write," Common Weakness Enumeration, 2019.

[32] MITRE Corporation, "CWE-125: Out-of-bounds Read," Common Weakness Enumeration, 2019.

[33] OWASP Foundation, "Input Validation Cheat Sheet," OWASP Cheat Sheet Series, 2023.

[34] OWASP Foundation, "Mobile Application Security Verification Standard (MASVS)," Version 2.0, 2023.

[35] Google Developers, "Android Security Internal Architecture and Application Sandbox," Android Open Source Project, 2023.

[36] Apple Inc., "Apple Platform Security Architecture Documentation," Apple Security Publications, 2023.

[37] Cybersecurity and Infrastructure Security Agency (CISA), "Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software," CISA/NSA/FBI Joint Guidance, 2023.

[38] CISA, "Known Exploited Vulnerabilities Catalog (KEV)," Cybersecurity and Infrastructure Security Agency, 2023.

[39] International Organization for Standardization, "Information Security Management Systems - Requirements," *ISO/IEC 27001:2022*, 2022.

[40] ISO/IEC, "Information Security Controls," *ISO/IEC 27002:2022*, 2022.